



TALK-TO-DATA (T2D)

Phase 4 Annexes: Design Architecture

Daniel Brulé

[LinkedIn](#) - [GitHub](#)

Version 1.0 - June 2026

Table of Contents

1. How to use this annex	2
2. Model patterns	3
2.1. Model family and likely use	3
2.2. Model use by orchestration task	3
3. Cost scenarios.....	5
3.1. Cost scenario worksheet	5
3.2. Indicative cost structure	5
3.3. Indicative cost-per-interaction template	5
3.4. Cost estimation template	6
4. Evaluation, model-switching and performance tracking.....	7
4.1. Simple model evaluation approach	7
4.2. Evaluation dimensions.....	7
4.3. Technology patterns for switching and tracking	7
5. Activities.....	9
5.1. Confirm orchestration principles and constraints	9
5.2. Question-to-answer flow worksheet.....	12
5.3. Metadata retrieval and grounding worksheet.....	15
5.4. Model, prompt and routing strategy worksheet.....	17
5.5. Tool boundaries and execution hand-off worksheet.....	18
5.6. Query generation and validation worksheet	20
5.7. Answer generation and safe-failure worksheet	22
5.8. Multi-turn conversation handling worksheet	24
5.9. Model and orchestration evaluation worksheet	27
5.10. Quality monitoring and improvement loop worksheet	30
5.11. Orchestration governance and change-control worksheet.....	33

1. How to use this annex

This annex pack contains example tools, templates and checklists supporting the **Phase 4 Design Architecture and Orchestration Guide**. It is intended as practical inspiration for facilitation, documentation and delivery, not as a mandatory to-do list.

The main guide explains the delivery logic and decisions required during Phase 4. This annex provides reusable working material that teams can adapt depending on the delivery stage, business risk, data maturity, technology stack, security requirements, model choices and governance requirements.

Teams should use only the templates that add value. For a narrow POC, many annex items may be simplified or skipped. For an MVP, pilot or production path, more of the material may be needed to support traceability, testing, security, cost control, ownership and operational handover.

The first three sections are cross-cutting. Model patterns, cost scenarios and evaluation / model-switching support Activities 5.3, 5.4, 5.9 and 5.10 in the main guide. The activity worksheets in Section 5 support specific Phase 4 design activities.

2. Model patterns

2.1. Model family and likely use

Model family	Example models / technologies	Likely use in T2D	Strengths	Watch-outs
Rules / deterministic logic	SQL rules, policy engines, regex, validation services	Access checks, routing rules, thresholds, validation gates	Cheap, auditable, predictable	Limited flexibility
Embedding model	text-embedding models, voyage embeddings, bge, e5	Metadata search, semantic matching, example retrieval	Low cost, fast, useful for grounding	Retrieval quality must be tested
Small / local / embedded model	GPT-4o mini, GPT-4.1 mini, Gemini Flash, Mistral Small, local 3B–8B models	Intent routing, metadata ranking, simple classification	Cheap, fast, easier to deploy locally	May fail on ambiguity
Medium general model	GPT-4.1, Claude Sonnet, Gemini Flash/Pro, Mistral Medium / Large	Query drafting, clarification, answer shaping	Good balance of cost and capability	Needs strong grounding and validation
Strong reasoning model	OpenAI reasoning models, Claude Opus, Gemini Pro reasoning models	Complex SQL, ambiguity resolution, hard edge cases	Better reasoning and robustness	Higher cost and latency
Self-hosted / private model	Llama, Mistral open-weight, Qwen, private cloud deployments	Sensitive data, residency constraints, regulated use cases	More control over deployment and data	Higher operating complexity
Fine-tuned / task-specific model	Fine-tuned small model, distilled model, classifier	Repeated narrow tasks with stable patterns	Consistent, efficient for known workflows	Requires data, maintenance and regression testing

2.2. Model use by orchestration task

Task	Likely model pattern	Notes
Intent classification	Rules, small model or medium model	Use stronger models only if intent is ambiguous or high risk.
Metadata retrieval	Embeddings plus small or medium model	Retrieval quality matters more than model size.
Metadata ranking	Small or medium model	Must be evaluated against approved artefacts.
Clarification question	Medium model	Should follow controlled clarification rules.
Query generation	Medium or strong model	Stronger model may be justified for complex SQL.

Query validation	Rules plus model-assisted review	Do not rely on the model alone for validation.
Result interpretation	Medium model	Must use query result, caveats and metric definitions.
Answer generation	Medium or strong model	Main risk is overstatement, missing caveats or wrong context.
Follow-up handling	Medium or strong model	Risk increases when context is inherited across turns.
Safety / refusal decision	Rules plus model-assisted classification	Refusal should be policy-led, not purely model-led.

3. Cost scenarios

3.1. Cost scenario worksheet

Cost should be estimated by orchestration step, not only as a total model bill. The same user interaction may trigger metadata retrieval, one or more model calls, query validation, execution, answer generation and logging.

Scenario	Initial request pattern	Follow-up pattern	Main cost drivers
Simple POC	One metadata retrieval, one model call for query, one execution, one answer call	Reuse context, limited new retrieval, one answer call	Model calls, small query cost, basic logging
MVP	Metadata retrieval, intent routing, query generation, validation, execution, answer generation	Context check, possible re-query, answer regeneration	Model routing, validation calls, query execution, logs
Complex pilot	Multi-step retrieval, stronger model for SQL, layered validation, execution, result interpretation, answer generation	Context inheritance, new filters, re-query, additional validation	Strong model use, warehouse cost, concurrency, trace retention

3.2. Indicative cost structure

Cost component	Initial request	Follow-up request	Notes
Metadata retrieval	Usually required	May be reused or refreshed	Use caching where safe.
Intent / routing model	Usually required	Usually required	Often suitable for a small model.
Query generation model	Required if query needed	Required if follow-up needs new query	Cost rises with schema/context size.
Query validation	Required before execution	Required before re-execution	Prefer deterministic checks where possible.
Query execution	Required if data is queried	May be avoided if prior result is enough	Can dominate cost at scale.
Answer generation	Required	Required	Main quality and safety risk.
Logging / trace storage	Required	Required	Retention policy affects cost.
Evaluation / regression	Periodic or batch	Periodic or batch	Should be budgeted from MVP onward.

3.3. Indicative cost-per-interaction template

The values below are placeholders for planning, not vendor quotes. They should be replaced with organisation-specific pricing, model choice, token volumes, warehouse cost, retrieval pattern, logging retention and expected usage.

Scenario	Retrieval cost	Model-call cost	Query execution cost	Logging / trace cost	Indicative total per initial interaction	Indicative total per follow-up
Simple POC	£0.001–£0.005	£0.01–£0.05	£0.001–£0.02	<£0.001	£0.01–£0.08	£0.005–£0.04
MVP	£0.003–£0.02	£0.03–£0.20	£0.005–£0.10	£0.001–£0.005	£0.04–£0.30	£0.02–£0.18
Complex pilot	£0.01–£0.05	£0.15–£1.00	£0.05–£1.00+	£0.005–£0.03	£0.20–£2.00+	£0.10–£1.50+

3.4. Cost estimation template

Input	Low scenario	Medium scenario	High scenario
Users	25	100	500
Interactions per user per day	3	5	10
Share of follow-up interactions	30%	50%	70%
Model calls per initial interaction	2	3	5
Model calls per follow-up	1	2	4
Average prompt size	2k tokens	6k tokens	15k tokens
Average answer size	300 tokens	700 tokens	1.5k tokens
Query executions per interaction	0.8	1.2	2.0
Average query execution cost	£0.005	£0.05	£0.50
Log retention period	30 days	90 days	180+ days
Evaluation runs per month	1	4	10+

Estimated monthly cost should include model calls, metadata retrieval, query execution, validation, logging, trace storage, evaluation, regression testing, monitoring and operational support.

4. Evaluation, model-switching and performance tracking

4.1. Simple model evaluation approach

Model evaluation should compare models on the same task, with the same inputs, against the same expected outputs. The evaluation should include both successful questions and failure cases.

Evaluation step	What to do
Define task	Evaluate one task at a time: retrieval, query generation, answer generation or follow-up handling.
Build test set	Include common questions, ambiguous questions, edge cases and out-of-scope questions.
Fix inputs	Use the same metadata, prompts, examples and user context for each model.
Run blind comparison	Compare outputs without favouring a provider or model family.
Score outputs	Use agreed dimensions, not personal preference.
Review failures	Identify whether failure came from retrieval, model reasoning, validation or answer generation.
Estimate cost and latency	Compare quality against cost and response time.
Select by task	Choose the simplest model pattern that meets quality, safety and cost thresholds.

4.2. Evaluation dimensions

Dimension	Question to answer
Retrieval quality	Did the model retrieve the right metadata, metric, dimension and examples?
Query correctness	Did the query use the right source, metric, filters, joins and grain?
Validation behaviour	Did the system block unsafe, invalid or unauthorised queries?
Answer correctness	Did the final answer match the trusted result?
Caveat handling	Were limitations, assumptions and caveats preserved?
Safe failure	Did the model clarify, refuse or escalate when needed?
Follow-up handling	Did it inherit the right context and avoid scope drift?
Consistency	Did repeated runs produce stable results?
Latency	Was the response fast enough for the user journey?
Cost	Was the model pattern sustainable for expected usage?
Data exposure	Was unnecessary data, metadata or query detail avoided?
Maintainability	Can prompts, models and evaluation sets be updated safely?

4.3. Technology patterns for switching and tracking

Teams should avoid hard-coding model choices directly into the orchestration logic. A simple abstraction layer makes it easier to compare models, switch providers, route tasks and track performance over time.

Technology / pattern	Where it helps	Watch-outs
Model gateway	Centralises provider access, routing, credentials, limits and fallback	Can become a bottleneck if poorly governed
LLM orchestration framework	Supports model calls, chains, tools, retrieval and tracing	Avoid framework lock-in or unnecessary complexity
Prompt / model registry	Tracks prompt versions, model versions and task ownership	Must be linked to evaluation and change control
Experiment tracking	Compares model outputs, latency, cost and quality scores	Needs consistent datasets and scoring rules
Observability platform	Tracks traces, errors, costs, latency and user feedback	Sensitive prompts and results need retention controls
Feature flags / routing rules	Allows controlled rollout of model or prompt changes	Requires rollback and approval process
Evaluation harness	Runs repeatable tests across models and prompts	Must include edge cases and failure scenarios

Example technologies may include LangChain, LlamaIndex, Semantic Kernel, Haystack, DSPy, MLflow, Weights & Biases, LangSmith, Helicone, PromptLayer, OpenTelemetry, provider-native tracing and cloud-native monitoring. The specific choice matters less than the capability: the team needs a controlled way to switch models, compare performance, trace failures and govern changes.

5. Activities

5.1. Confirm orchestration principles and constraints

For model, cost and evaluation assumptions that cut across activities, see Sections 2 to 4.

5.1.1. Orchestration principles checklist

Principle	Design question	Evidence to capture
Governed context first	Does the model use approved metadata before generating queries or answers?	Metadata sources, retrieval rules, grounding approach.
Model judgement is constrained	What must not be decided by the model alone?	Rules for metrics, joins, filters, access and caveats.
Tool use is bounded	Which tools can be called, and under what limits?	Tool list, permissions, limits and blocked actions.
Validate before action	What is checked before query execution or response generation?	Validation rules, dry-run approach, refusal criteria.
Fail safely	When should the system clarify, refuse or escalate?	Safe-failure rules and examples.
Explain the answer	What must be visible to support trust?	Sources, definitions, caveats, assumptions and query evidence.
Log what matters	Can failures, misuse and quality issues be investigated?	Required logs, traces and retention assumptions.
Govern change	Who approves changes to models, prompts, tools and validation?	Change owners, approval route and decision log.

5.1.2. Security, exposure, cost and scale assumptions template

Area	Question
User identity	How is the user identified?
Access enforcement	Where are permissions enforced?
Row-level security	Are row-level rules inherited or recreated?
Column restrictions	Which fields must be hidden, masked or excluded?
Sensitive data	Can personal, financial or regulated data reach the model?
Metadata exposure	Which schema, metric and example metadata can be shared?
Query text	Can generated SQL or tool calls be sent to the model?
Result data	Can query results be passed back to the model for explanation?
Logs and traces	What is logged, retained and searchable?
Audit needs	What evidence must be available for audit or investigation?
Data residency	Are there location or platform constraints?
Retention	How long are prompts, responses, logs and feedback retained?
Model deployment	Where can the model run: EU, US, China, private cloud, on-premise or approved SaaS?
Model type	Are open, closed, hosted, private or fine-tuned models allowed for this use case?

Cross-border transfer	Can prompts, metadata, query text, results, logs or feedback leave the required region?
Cost envelope	What cost range is acceptable for POC, MVP and pilot?
Cost drivers	Which costs need active control: model calls, tokens, retrieval, query execution, logging or evaluation?
Usage limits	Are there limits by user, question, session, query size or time period?
Cost alerts	What thresholds should trigger review or throttling?
User scale	How many users are expected in POC, MVP, pilot and production?
Concurrency	How many simultaneous sessions or questions should be supported?
Usage pattern	Are questions expected to be ad hoc, daily, peak-period or event-driven?
Response time	What latency is acceptable by question type or user group?
Query load	What query volume can the warehouse, API or semantic layer support?
Throttling	When should usage be slowed, queued, blocked or escalated?

5.1.3. Model and tool constraint register

Constraint area	Decision to capture	Example
Model role	Which tasks can the model perform?	Intent routing, query drafting, answer explanation.
Model limits	What must the model not decide?	Metric logic, access rights, unsafe joins.
Prompt scope	What context can be included in prompts?	Approved metric cards, schema snippets, caveats.
Tool access	Which tools can the orchestration call?	Metadata search, query validation, SQL execution.
Tool permissions	What is each tool allowed to do?	Read-only query execution with row limits.
Blocked actions	What is explicitly prohibited?	Write-back, unrestricted SQL, cross-domain joins.
Human review	Which actions require approval or escalation?	Sensitive data, high-risk answers, failed validation.
Logging	What must be logged for each model or tool step?	Prompt ID, metadata version, query, validation result.

5.1.4. Change-control decision log

Change type	Example	Approval owner	Evidence required
Model change	Change SQL-generation model.	AI / solution architect.	Evaluation comparison and risk review.
Prompt change	Update query-generation prompt.	AI / solution architect.	Regression test results.
Tool change	Add a new execution tool.	Security / governance lead.	Permission review and logging design.
Metadata contract change	Add new fields to metric cards.	Data / semantic owner.	Updated register and version history.
Validation change	Add or relax a validation rule.	Evaluation owner / governance lead.	Failure analysis and test evidence.

Threshold change	Change cost, latency or quality thresholds.	Product owner / operating owner.	Impact assessment.
Safe-failure change	Change refusal or escalation rules.	Product owner / governance lead.	User and risk review.

5.1.5. Minimum evidence before closing Activity 5.1

Before closing this activity, the team should be able to show:

- security, exposure and deployment assumptions;
- cost, latency and scale assumptions;
- model and tool-use constraints;
- logging and audit assumptions;
- change-control expectations;

5.2. Question-to-answer flow worksheet

5.2.1. End-to-end flow worksheet

The flow is shown sequentially for clarity, but it may loop. Query generation and validation may iterate when the query needs revision, metadata is missing, permissions fail, or the user must clarify the request. The loop should be bounded: after defined attempts or failed checks, the system should refuse, clarify or escalate rather than continue regenerating.

Step	Purpose	Main input	Main output	Owner	Evidence captured
User question received	Capture the user request and context.	User question, user identity, session context.	Normalised request.	Product / orchestration owner	Question, timestamp, user/session ID.
Intent and scope check	Decide whether the question is in scope.	Normalised request, approved scope.	In-scope, clarify, refuse or escalate.	Orchestration owner	Intent, scope decision, reason.
Context handling	Decide what prior context is relevant.	Current question, session history.	Context to retain or discard.	Orchestration owner	Inherited filters, period, metric, entity.
Metadata retrieval	Retrieve governed context.	Question, scope, metadata contract.	Relevant metrics, dimensions, joins, caveats.	Data / semantic owner	Metadata IDs, versions, retrieval result.
Query planning	Decide whether a query is needed.	Intent, metadata, context.	Query plan or no-query response route.	AI / solution architect	Query intent, selected sources, assumptions.
Query generation	Generate a query or tool call using approved metrics, joins, filters and limits	Query plan, metadata, user context, constraints .	Draft query or tool call.	AI / solution architect	Prompt/version , generated query/tool call.
Query validation	Check syntax, scope, permissions , joins, filters, row limits, cost and risk before execution	Draft query, validation rules, permission	Approve, revise, block, clarify or escalate	Evaluation / governance owner	Validation checks, failures, revision count, final decision

Execution hand-off	Execute through approved route.	Validated query or tool call.	Query result or execution failure.	Data / platform owner	Execution ID, duration, cost, row count.
Result interpretation	Interpret result against context and caveats.	Query result, metadata, caveats.	Interpreted result.	Orchestration owner	Result checks, caveats applied.
Answer generation	Produce the user-facing answer.	Interpreted result, response rules.	Answer, caveat, refusal or escalation.	Orchestration owner	Answer, sources, assumptions, confidence signals.
Feedback and logging	Capture feedback and trace.	Answer, logs, user response.	Trace, feedback, improvement signal.	Operating owner	Feedback, trace ID, issue category.

5.2.2. Flow decision checklist

Decision area	Question	Decision / assumption
Scope	What questions are allowed in the first flow?	
Clarification	When should the system ask a clarification question?	
Refusal	When should the system refuse rather than answer?	
Escalation	When should a human or analyst review be triggered?	
Context	Which parts of prior conversation can be reused?	
Metadata	Which governed artefacts must be retrieved before query generation?	
Query	When is query generation allowed?	
Validation	Which checks must pass before execution?	
Execution	Which route is used for execution: SQL, API, semantic layer or BI tool?	
Answer	What must be shown with the answer: source, caveat, period, metric or SQL?	
Logging	What must be logged at each step?	
Feedback	How can users flag wrong, unclear or unhelpful answers?	

5.2.3. Safe-failure examples

Situation	Preferred behaviour	Evidence to capture
User asks for an unsupported metric	Refuse or explain scope; suggest supported alternatives.	Question, unsupported metric, refusal reason.
User question is ambiguous	Ask a clarification question before querying.	Ambiguity type, clarification asked.
Metadata retrieval fails	Do not generate query; escalate or ask for support.	Retrieval failure, missing artefact.
User lacks permission	Refuse or return permitted aggregate only.	Permission check, denied scope.

Query validation fails	Block execution and explain limitation if appropriate.	Failed rule, query version, owner.
Result conflicts with caveat	Answer with caveat or escalate for review.	Caveat applied, result issue.
Follow-up changes scope silently	Restate assumptions or ask clarification.	Prior context, new context, decision.

5.2.4. Minimum evidence before closing Activity 5.2

Before closing this activity, the team should be able to show:

- an agreed question-to-answer flow;
- main decision points and failure paths;
- clarification, refusal and escalation rules;
- validation and execution sequence;
- logging points across the flow;
- prototype boundaries and simplifications.

5.3. Metadata retrieval and grounding worksheet

5.3.1. Metadata contract checklist

Metadata area	Required fields	Runtime use
Metric	ID, name, synonyms, definition, formula, owner, status, version, caveats	Select metric, generate query, explain answer
Dimension	ID, name, synonyms, allowed values, hierarchy, owner, status, version	Filter, group, drill down, clarify
Join rule	ID, tables, keys, grain, cardinality, allowed use, caveats	Prevent unsafe joins and duplication
Filter rule	ID, default filters, exclusions, date logic, mandatory conditions	Apply standard business logic
Queryable asset	ID, table/view/API, owner, status, grain, access rules, freshness	Select allowed source
Example question	Question, expected metric, filters, query pattern, answer shape	Guide query generation and evaluation
Security rule	Role, row/column restrictions, masking, aggregation limits	Enforce access and exposure controls
Quality limit	Freshness, completeness, known gaps, reliability caveats	Decide whether to answer, caveat or refuse

5.3.2. Retrieval design worksheet

Where retrieval or ranking uses a model, the model pattern, cost and evaluation assumptions should align with Sections 2 to 4.

Design area	Question	Decision / assumption
Retrieval source	Where will metadata be retrieved from?	
Retrieval timing	Is metadata retrieved at runtime, build time or both?	
Search method	Is retrieval keyword-based, semantic, rules-based or hybrid?	
Ranking	How are competing metadata results ranked?	
Versioning	How is the approved version identified?	
Scope filtering	How are out-of-scope assets excluded?	
Access filtering	Is metadata filtered by user permission or role?	
Conflict handling	What happens when definitions conflict?	
Missing metadata	When should the system clarify, refuse or escalate?	
Logging	Which metadata IDs and versions are logged?	
Evaluation	How is retrieval quality tested?	

5.3.3. Grounding rules

Grounding area	Rule to define	Example decision
Query generation	What metadata must be present before query generation?	Metric ID, source asset, grain, join rule and filters required.

Validation	Which metadata is checked before execution?	Approved joins, access rules, row limits and mandatory filters.
Answer generation	Which metadata must appear in the answer or caveat?	Metric definition, period, caveat and source.
Follow-up handling	Which metadata can be inherited across turns?	Metric and period may persist; new dimension requires validation.
Refusal	Which metadata gaps should block an answer?	No approved metric or unsafe join path.

5.3.4. Missing or conflicting metadata handling

Situation	Preferred behaviour	Evidence to capture
Metric not found	Ask clarification or refuse unsupported metric.	User term, retrieval result, decision.
Multiple definitions found	Ask clarification or use approved scope rule.	Candidate IDs, ranking, chosen rule.
Join rule missing	Block query and route to remediation.	Tables, missing join, owner.
Caveat missing	Do not present answer as fully trusted.	Metric/source, missing caveat.
Metadata stale	Warn, refuse or escalate depending on risk.	Version, date, status.
Access rule unclear	Block execution until rule is confirmed.	User role, asset, unresolved rule.

5.3.5. Minimum evidence before closing Activity 5.3

Before closing this activity, the team should be able to show:

- required metadata fields for orchestration;
- retrieval source and timing;
- grounding rules for query generation, validation and answers;
- versioning and conflict-handling rules;
- missing-metadata behaviours;
- logging requirements for metadata IDs and versions.

5.4. Model, prompt and routing strategy worksheet

5.4.1. Model/task allocation

Task	Selected model / pattern	Reason	Constraints	Fallback
Intent routing				
Metadata ranking				
Query generation				
Query validation support				
Answer generation				
Follow-up handling				
Safe-failure classification				

5.4.2. Prompt control checklist

Area	Question	Decision / assumption
Prompt ownership	Who owns each prompt?	
Prompt versioning	How are prompt versions tracked?	
Prompt inputs	What data and metadata can enter the prompt?	
Prompt exclusions	What must never be included?	
Prompt testing	Which regression tests must pass after changes?	
Prompt logging	Which prompt ID, version and inputs are logged?	
Prompt approval	Who approves changes before release?	

5.4.3. Routing and fallback rules

Situation	Preferred route	Fallback / escalation
Simple intent	Rules or small model	Medium model if unclear
Metadata ambiguity	Retrieval plus ranking	Ask clarification
Complex SQL	Medium or strong model	Escalate if validation fails
Validation failure	Deterministic block	Revise once or escalate
High-risk answer	Stronger review path	Human review or refusal
Model unavailable	Approved fallback model	Degraded mode or pause

5.4.4. Minimum evidence before closing Activity 5.4

Before closing this activity, the team should be able to show:

- model/task allocation;
- prompt ownership and versioning;
- routing and fallback rules;
- model deployment and retention assumptions;

- cost and latency assumptions;
- evaluation approach for shortlisted models.

5.5. Tool boundaries and execution hand-off worksheet

5.5.1. Tool registry

Tool	Purpose	Allowed actions	Prohibited actions	Identity model	Owner
Metadata retrieval tool	Retrieve approved metadata.	Search/read approved artefacts.	Edit metadata or retrieve out-of-scope assets.	Service or user-context filtered.	Data / semantic owner
Query validation tool	Check query safety before execution.	Parse, dry-run, check rules and limits.	Execute unapproved queries.	Service identity.	Evaluation / governance owner
Query execution tool	Run approved query or API call.	Read-only execution against approved route.	Write-back, DDL, unrestricted SQL.	User or delegated identity.	Data / platform owner
Logging / trace tool	Capture orchestration evidence.	Store trace, decisions, IDs and outcomes.	Store unnecessary sensitive data.	Service identity.	Operating owner
Feedback tool	Capture user feedback.	Record rating, issue type and comments.	Override answer quality status without review.	User identity.	Product / operating owner

5.5.2. Tool permission worksheet

Decision area	Question	Decision / assumption
Identity	Does the tool run under user identity, service identity or delegated access?	
Scope	Which datasets, metadata, APIs or environments can the tool access?	
Access inheritance	Does the tool inherit existing row, column and masking controls?	
Input limits	What input size, query shape or prompt content is allowed?	
Output limits	What row count, fields, result size or detail level can be returned?	
Cost limits	What scan, token, query or API cost thresholds apply?	
Timeout limits	When should the tool stop, retry, queue or fail?	
Environment	Can the tool run in dev, test, pilot and production?	
Logging	What input, output, decision and error details are logged?	

Escalation	What failures require human review or support?	
-------------------	--	--

5.5.3. Execution hand-off pattern

Step	Control question	Required evidence
Prepare tool request	Is the request scoped, authorised and complete?	User/session ID, metadata IDs, selected tool.
Validate input	Is the input safe, allowed and within limits?	Input checks, blocked fields, size limits.
Execute tool call	Is the call routed through the approved execution path?	Tool ID, execution route, timestamp.
Validate output	Is the result safe to return or pass to the model?	Row count, masked fields, caveats, risk checks.
Handle failure	Should the system retry, clarify, refuse or escalate?	Error type, retry count, final decision.
Log trace	Can the action be reconstructed later?	Trace ID, inputs, outputs, decisions, owner.

5.5.4. Tool failure handling

Failure	Preferred behaviour	Evidence to capture
Metadata tool returns no result	Ask clarification, refuse or route to metadata remediation.	Search terms, missing artefact, decision.
Validation tool fails	Block execution and route to review if needed.	Failed rule, query version, owner.
Execution tool times out	Retry within limits or explain temporary failure.	Timeout, retry count, duration.
Execution cost exceeds threshold	Block or require approval.	Estimated cost, threshold, decision.
Result exceeds row limit	Aggregate, restrict, paginate or refuse.	Row count, limit, applied action.
Tool returns restricted fields	Mask, block or escalate.	Field names, policy rule, decision.
Logging tool fails	Continue only if minimum audit evidence is preserved.	Missing log, fallback trace, owner.

5.5.5. Minimum evidence before closing Activity 5.5

Before closing this activity, the team should be able to show:

- approved tool registry;
- identity and permission model;
- read/write and execution limits;
- timeout, row-limit and cost controls;
- tool failure behaviours;
- logging requirements for tool calls;
- unresolved tool risks and owners.

5.6. Query generation and validation worksheet

The examples in this annex are illustrative. They should be adapted to the actual use case, data sensitivity, user scope, technology stack, delivery stage and organisational controls. The aim is to provide a practical starting point, not a mandatory query policy.

5.6.1. Applying approved metrics, joins, filters and limits

Design area	Concrete example	Expected control
Approved metric	User asks for “revenue”; system maps to approved net_revenue metric.	Use metric ID, approved formula, owner and caveats from metadata contract.
Approved dimension	User asks “by region”; system maps to approved customer or sales region dimension.	Use only approved dimensions for the selected metric.
Approved join	Revenue by product uses approved order-line to product join.	Reject unapproved joins or many-to-many joins without a rule.
Date logic	“Last month” maps to approved fiscal or calendar period rule.	Apply approved date rule and show period used.
Standard filters	Revenue excludes cancelled orders and test accounts.	Apply mandatory filters automatically.
Row limit	User asks for customer-level breakdown.	Apply row limit, aggregation threshold or refusal rule.
Cost limit	Query scans more data than allowed.	Dry-run, block or require approval.
Access rule	User asks for region they cannot access.	Enforce row/column security before execution.

5.6.2. Query type policy

Query type	Status	Example	Expected behaviour
Simple aggregate	Allowed	Revenue last month.	Generate, validate and execute.
Grouped aggregate	Allowed	Revenue by region.	Execute if dimension is approved.
Trend query	Allowed	Monthly revenue for the last 12 months.	Execute with approved date logic.
Drill-down	Restricted	Revenue by customer.	Allow only if permissions and row limits pass.
Cross-domain join	Restricted	Revenue by support ticket category.	Allow only if approved join path exists.
Free-text SQL request	Restricted	“Run this SQL.”	Validate against policy or refuse.
Forecasting / prediction	Restricted	“Forecast next quarter revenue.”	Route to supported analytical workflow or refuse.
Write / update	Blocked	Update customer status.	Refuse; T2D should be read-only.
DDL / destructive SQL	Blocked	Drop table, create table, alter view.	Block deterministically.
Unbounded detail export	Blocked	Export all customer transactions.	Refuse or route to governed export process.

5.6.3. Validation checks before execution

Validation check	Example failure	Expected action
Syntax check	Invalid SQL.	Revise within limits or block.
Scope check	Question outside approved use case.	Refuse or escalate.
Permission check	User lacks access to region, product or table.	Block or restrict result.
Source check	Query uses unapproved table.	Block.
Metric check	Query uses unapproved calculation.	Revise or block.
Join check	Query uses unsafe many-to-many join.	Block or require approved join rule.
Filter check	Mandatory exclusions missing.	Revise query.
Grain check	Aggregation mixes incompatible grains.	Block or revise.
Row-limit check	Result may return too many rows.	Aggregate, limit or block.
Cost check	Query exceeds scan or compute threshold.	Dry-run, block or require approval.
Sensitive-field check	Query selects restricted fields.	Mask, remove or block.
Result-shape check	Output is too detailed for answer use.	Aggregate, summarise or block.
Runtime estimate	Query is expected to exceed runtime or timeout threshold.	Block, simplify, ask user to narrow scope or require approval.

5.6.4. Revise, clarify, refuse or escalate rules

Situation	Preferred action	Example response behaviour
Query syntax fails	Revise once or twice, then block.	"I could not generate a valid query for this request."
User term is ambiguous	Clarify.	"Do you mean gross revenue or net revenue?"
Metric not approved	Refuse or suggest supported metric.	"I cannot answer using an unapproved revenue definition."
Join path missing	Refuse or escalate to data owner.	"This breakdown is not currently supported."
Permission fails	Refuse or return authorised aggregate.	"You do not have access to that level of detail."
Query cost too high	Simplify, ask user to narrow, or require approval.	"Please narrow the time period or grouping."
Result too detailed	Aggregate or refuse.	"I can provide a summary, not row-level records."
Validation repeatedly fails	Stop loop and escalate.	"This question needs review before it can be answered."
Runtime estimate too high	Ask user to narrow scope or reduce detail.	"This query may take too long. Please reduce the time period, filters or level of detail."

5.6.5. Validation approach

Validation layer	Best handled by	Notes
Block write / delete / drop commands	Deterministic rules	Should never rely on model judgement.
Restrict tables and columns	Deterministic rules	Use allowlists, denylists and permissions.
Enforce row limits and timeouts	Deterministic rules	Apply before execution.
Check approved metrics and joins	Rules plus metadata checks	Use Phase 3 metadata contract.
Detect ambiguous business terms	Model-assisted	Should trigger clarification.
Review complex SQL intent	Model-assisted plus rules	Useful for harder queries, but not sufficient alone.
Assess answer caveats	Model-assisted plus metadata	Must preserve approved caveats.

5.6.6. Minimum evidence before closing Activity 5.6

Before closing this activity, the team should be able to show:

- approved query-generation constraints;
- allowed, restricted and blocked query types;
- validation checks before execution;
- revise, clarify, refuse and escalation rules;
- deterministic controls for obvious unsafe actions;
- logging requirements for query versions and validation results;
- unresolved query risks and owners.

5.7. Answer generation and safe-failure worksheet

5.7.1. Answer content checklist

Answer element	Required / optional	Example	Notes
Direct answer	Required	“Revenue last month was £4.2m.”	Should match the query result.
Metric definition	Required where relevant	“Revenue means net revenue excluding cancelled orders.”	Use approved metadata.
Period used	Required	“Period: March 2026.”	Avoid hidden date assumptions.
Filters applied	Required where relevant	“Region: UK; channel: online.”	Show material filters.
Source	Required	“Source: governed sales mart.”	Use approved source name.

Caveats	Required where relevant	“Excludes late adjustments.”	Preserve approved caveats.
Assumptions	Required where relevant	“Assumed calendar month, not fiscal period.”	Clarify if high risk.
SQL / query	Optional	“Show SQL.”	Useful for technical users; governed access needed.
Confidence signal	Optional	“Based on approved metric and current data.”	Avoid false precision.
Suggested follow-up	Optional	“Would you like this by region?”	Should stay within approved scope.

5.7.2. Result interpretation checks

Check	Example issue	Expected action
Empty result	No rows returned.	Explain no result or ask user to narrow/change scope.
Partial result	Data missing for one region.	Caveat or escalate.
Stale data	Data last refreshed 10 days ago.	Show freshness caveat or refuse if too stale.
Unexpected spike	Revenue increases 300%.	Caveat and suggest validation or analyst review.
Suppressed detail	Row-level result blocked by policy.	Return permitted aggregate or refuse.
Conflicting caveat	Result conflicts with known limitation.	Apply caveat or escalate.
Low retrieval confidence	Metric or source match uncertain.	Clarify before answering.

5.7.3. Refusal, clarification and escalation rules

Situation	Preferred behaviour	Example response pattern
Ambiguous metric	Clarify.	“Do you mean gross revenue or net revenue?”
Unsupported question	Refuse and suggest supported scope.	“This question is outside the approved T2D scope.”
Missing approved metric	Refuse or escalate.	“I cannot answer without an approved metric definition.”
Unsafe join or query path	Refuse or route to remediation.	“This breakdown is not currently supported.”
Permission failure	Refuse or restrict.	“You do not have access to that level of detail.”
High-risk result	Escalate or caveat.	“This result needs review before use in decision-making.”
Follow-up changes scope	Clarify or restate assumptions.	“Do you want to keep the same region and period?”

5.7.4. Evidence visibility options

User type	Recommended evidence	Watch-outs
Executive user	Definition, period, source, caveats.	Avoid overwhelming detail.

Business manager	Definition, filters, source, caveats, freshness.	Keep explanation readable.
Analyst	Definition, filters, source, caveats, SQL/query.	SQL visibility must respect permissions.
Auditor / governance reviewer	Metadata IDs, query, validation result, logs, versions.	Usually not shown in normal user answer.

5.7.5. Answer safety checks before response

Safety check	Question	Action if failed
Metric check	Is the approved metric definition used?	Block or revise.
Source check	Is the approved source shown?	Add source or block.
Period check	Is the time period explicit?	Add period or clarify.
Caveat check	Are required caveats preserved?	Add caveat or block.
Permission check	Does the answer expose restricted information?	Mask, aggregate or refuse.
Certainty check	Is the wording stronger than the evidence?	Soften wording or caveat.
Follow-up check	Are inherited assumptions visible?	Restate assumptions or clarify.

5.7.6. Minimum evidence before closing Activity 5.7

Before closing this activity, the team should be able to show:

- answer content rules;
- metadata and evidence visibility rules;
- result interpretation checks;
- refusal, clarification and escalation rules;
- answer safety checks;
- SQL/query visibility position;
- logging requirements for answers and refusal reasons.

5.8. Multi-turn conversation handling worksheet

5.8.1. Context inheritance rules

Context inheritance should not mean passing the whole conversation back to the model. A safer pattern is to maintain a small, structured session state that records only the context needed for follow-up questions. Each new turn should update, reuse, reset or reject that state.

Context element	How to implement	Example	Required control
Metric	Store selected metric ID and label in session state.	metric_id = net_revenue	Revalidate if user changes metric.
Time period	Store explicit period and period type.	period = Mar 2026, type = calendar month	Restate period in answer.
Filters	Store filter fields and values.	region = UK, channel = online	Show material filters.

Dimension	Store current grouping only if approved.	group_by = region	Revalidate when dimension changes.
Entity	Store entity ID or label where needed.	customer = ABC Ltd	Recheck access before reuse.
Prior result	Store result reference, not full raw output where possible.	result_id = R12345	Check freshness and scope.
Caveats	Store caveat IDs linked to metadata.	caveat_id = late_adjustments_excluded	Preserve in follow-up answer.
Access context	Store role/access decision reference.	access_profile = sales_uk_manager	Recheck for new data/detail.
User preference	Store only low-risk preferences.	format = chart	Do not store sensitive preferences.
Sensitive detail	Avoid storing unless essential.	customer-level rows	Mask, aggregate or discard.

The system should treat context as structured state, not memory. Each follow-up should classify whether it reuses existing context, modifies it, requires new metadata, requires a new query, or must be clarified/refused. Material inherited assumptions should be shown back to the user.

Example

User turn	State change	System behaviour
“What was revenue last month?”	Set metric = revenue; period = last month.	Retrieve metadata, query, answer with period and definition.
“Break it down by region.”	Reuse metric and period; add dimension = region.	Validate region dimension and re-query.
“What about margin?”	Metric change detected.	Retrieve margin metadata or clarify if multiple definitions exist.
“Show customer-level detail.”	Detail level change detected.	Recheck permissions and row/detail limits.

5.8.2. Follow-up handling patterns

Follow-up type	Example	Preferred behaviour
Drill-down	“Break that down by region.”	Reuse metric and period, validate dimension, re-query.
Filter change	“Only for the UK.”	Apply new filter, restate assumptions, re-query.
Period change	“What about last quarter?”	Replace period, re-query.
Comparison	“Compare that to last year.”	Keep metric, add comparison period, re-query.
Explanation	“Why did it change?”	Use supported drivers only; clarify if root-cause data is not approved.
Format change	“Show as a chart.”	Reuse result if still valid.
Scope expansion	“Now include customer-level detail.”	Check permission and row/detail limits.

Ambiguous follow-up	“What about margin?”	Clarify metric or scope before answering.
Out-of-scope follow-up	“Forecast next quarter.”	Refuse or route to supported workflow.

5.8.3. Clarify, re-query, reuse or refuse rules

Situation	Preferred action	Reason
Follow-up keeps same metric and period	Reuse or re-query depending on request.	Avoid unnecessary cost but keep accuracy.
Follow-up changes metric	Clarify or retrieve new metadata.	Metric logic may differ.
Follow-up changes dimension	Revalidate and usually re-query.	Dimensions may have different joins or limits.
Follow-up adds restricted detail	Check permissions before answering.	Prevent exposure through conversation.
Prior result is stale or partial	Re-query or caveat.	Avoid answering from weak evidence.
Context is ambiguous	Clarify.	Prevent hidden assumptions.
Follow-up moves outside approved scope	Refuse or escalate.	Keep system within governed use case.
Multiple failed turns occur	Escalate or end safely.	Avoid repeated guessing.

5.8.4. Session retention and logging

Area	Design question	Decision / assumption
Session duration	How long is conversation context retained?	
Context summary	Is full history retained or summarised?	
Sensitive context	What must not be retained across turns?	
User identity	How is identity linked to the session?	
Permission recheck	When are permissions revalidated?	
Context log	Which inherited assumptions are logged?	
Refusal log	Are refusals and reasons captured?	
Feedback link	Can feedback be linked to the relevant turn?	
Retention policy	How long are conversation traces retained?	

5.8.5. Conversation safety checks

Safety check	Question	Action if failed
Scope check	Is the follow-up still in approved scope?	Refuse or escalate.
Context check	Are inherited assumptions still valid?	Restate, clarify or reset.
Permission check	Does the user still have access?	Block, mask or aggregate.
Freshness check	Is the prior result still usable?	Re-query or caveat.
Detail check	Does the follow-up request restricted detail?	Refuse or restrict.

Safety check	Question	Action if failed
Caveat check	Are prior caveats still preserved?	Restate caveats.
Ambiguity check	Could the follow-up mean several things?	Clarify.

5.8.6. Minimum evidence before closing Activity 5.8

Before closing this activity, the team should be able to show:

- context inheritance rules;
- follow-up handling patterns;
- clarify, re-query, reuse and refusal rules;
- session retention assumptions;
- conversation safety checks;
- logging requirements for context and turn-level decisions.

5.9. Model and orchestration evaluation worksheet

5.9.1. Evaluation tool options

Tool / pattern	Where it helps	Pros	Cons
Spreadsheet scorecard	Early manual evaluation of model outputs, answers and refusals.	Simple, fast, accessible to business reviewers.	Hard to scale, weak traceability, manual effort.
SQL / BI comparison	Compare generated results against trusted reports or benchmark queries.	Strong for numeric correctness and reconciliation.	Does not assess explanation quality or safe failure.
Golden test set	Repeatable set of approved questions, expected queries and expected answers.	Good baseline for regression and model comparison.	Needs maintenance as metrics, data and scope change.
Human review panel	SMEs review answer usefulness, caveats and business interpretation.	Captures judgement and trust issues.	Slow, subjective, needs clear scoring rubric.
Automated unit tests	Test deterministic rules, blocked SQL, permissions and validation gates.	Fast, repeatable, CI/CD friendly.	Does not cover nuanced language or judgement.
LLM-as-judge	Model reviews outputs against rubric or reference answer.	Useful for scale and qualitative checks.	Must be validated; can introduce bias or false confidence.
Prompt / model registry	Tracks prompt versions, model versions and evaluation results.	Supports governance and rollback.	Needs discipline and ownership.
Experiment tracking	Compares models, prompts, cost, latency and scores.	Good for structured model selection.	Requires consistent datasets and metrics.

Observability / tracing platform	Tracks traces, failures, latency, cost and feedback.	Essential for debugging and monitoring.	Sensitive prompts/results need retention controls.
Evaluation framework	Runs repeatable evals across models, prompts and datasets.	Useful for regression and model switching.	Can be over-engineered for early POCs.

5.9.2. Example technologies

Technology	Likely use	Pros	Cons
LangSmith	Tracing, evaluation, dataset management for LangChain-based flows.	Strong tracing and prompt/model comparison.	Most useful if using LangChain ecosystem.
LangChain	Orchestration, tool calling, retrieval and evaluation hooks.	Broad ecosystem and quick prototyping.	Can add complexity and framework lock-in.
LlamaIndex	Retrieval, metadata grounding and evaluation around data connectors.	Strong for retrieval-heavy workflows.	Less suited if orchestration is not retrieval-centred.
Semantic Kernel	Enterprise-oriented orchestration, especially in Microsoft environments.	Fits Azure / Microsoft stack well.	Less neutral if not using Microsoft ecosystem.
MLflow	Experiment tracking, model registry and evaluation logging.	Familiar MLOps pattern, strong governance fit.	Less specialised for LLM traces and conversations.
Weights & Biases	Experiment tracking, evaluation and model comparison.	Strong experiment management.	May be heavier than needed for business POCs.
Ragas	Retrieval and RAG evaluation.	Useful for retrieval quality and grounding.	Needs careful adaptation for T2D, not generic RAG only.
DeepEval	LLM evaluation tests and metrics.	Good for automated test suites.	Metrics must be customised to T2D risks.
Promptfoo	Prompt and model regression testing.	Lightweight and practical for comparing prompts/models.	Less complete for full production observability.
OpenTelemetry	Standardised tracing across systems.	Vendor-neutral and production-friendly.	Requires engineering setup.
Helicone	LLM observability, cost and latency tracking.	Quick visibility into model calls.	Depends on deployment and routing pattern.
Provider-native tools	OpenAI, Anthropic, Azure, Google or AWS tracing/evaluation tools.	Integrated with provider stack.	May increase provider lock-in.

5.9.3. Evaluation design template

Area	Decision / question	Example
------	---------------------	---------

Evaluation scope	Which tasks are evaluated?	Retrieval, SQL generation, answer generation, follow-ups.
Test set	Which questions are included?	Common, ambiguous, edge-case and out-of-scope questions.
Reference answer	What is the expected answer?	Trusted SQL result, approved caveats, expected refusal.
Scoring method	How is quality scored?	1–5 scale with pass/fail blockers.
Human review	Who reviews business quality?	SME, data owner, product owner.
Automated checks	Which checks are automated?	SQL validity, blocked commands, row limits, source use.
LLM judge use	Where is model review acceptable?	Caveat preservation, answer clarity, refusal quality.
Thresholds	What score is required to proceed?	No critical failure; minimum score by task.
Regression	When are tests rerun?	Prompt, model, metadata or validation changes.
Evidence	What is retained?	Test ID, prompt version, model version, output, score, reviewer.

5.9.4. Suggested scoring dimensions

Dimension	What to score	Blocking failure example
Retrieval quality	Correct metadata, metric, source, examples.	Wrong metric retrieved.
Query correctness	Correct source, joins, filters, grain and period.	Unsafe join reaches execution.
Validation behaviour	Unsafe or invalid actions are blocked.	Write/delete/drop query is allowed.
Answer correctness	Final answer matches trusted result.	Wrong number or unsupported conclusion.
Caveat handling	Caveats and assumptions are preserved.	Required caveat omitted.
Safe failure	Clarifies, refuses or escalates when needed.	Answers unsupported question.
Follow-up handling	Correct context inheritance and revalidation.	User moves outside scope silently.
Security / exposure	Avoids unnecessary data exposure.	Sensitive data included in prompt or answer.
Latency	Response time fits user journey.	Answer too slow for intended use.
Cost	Cost fits usage assumptions.	Uses strongest model for every step.
Usability	Answer is understandable and useful.	Technically correct but unclear to user.

5.9.5. Tool selection guidance

Situation	Preferred approach
Early POC	Spreadsheet scorecard, golden test set, manual SME review, provider logs.
MVP	Golden test set, automated validation tests, prompt/model registry, tracing platform.

Pilot	Evaluation framework, observability platform, regression tests, feedback-linked monitoring.
Production path	CI/CD evaluation gates, model registry, OpenTelemetry or equivalent tracing, formal approval workflow.

5.9.6. Minimum evidence before closing Activity 5.9

Before closing this activity, the team should be able to show:

- evaluation scope and test set;
- scoring dimensions and thresholds;
- model and prompt comparison approach;
- tooling selected for evaluation and tracing;
- human and automated review responsibilities;
- regression trigger rules;
- retained evaluation evidence and owners.

5.10. Quality monitoring and improvement loop worksheet

5.10.1. Quality signal catalogue

Signal	What it shows	Example metric
User feedback	Whether users trust or find the answer useful.	Thumbs down rate by question type.
Answer correction	Where answers were wrong or incomplete.	Number of corrected answers per week.
Failed validation	Where generated queries or tool calls are blocked.	Validation failure rate by rule.
Refusals	Where the system cannot or should not answer.	Refusal rate by reason.
Clarifications	Where user questions are ambiguous.	Clarification rate by metric or domain.
Escalations	Where human review is needed.	Escalations by risk category.
Retrieval failures	Where metadata grounding is weak.	Missing or conflicting metadata rate.
Query failures	Where execution fails or times out.	Query failure and timeout rate.
Latency	Whether response time fits the user journey.	Median and 95th percentile response time.
Cost	Whether usage is financially sustainable.	Cost per interaction and cost per active user.
Drift	Whether quality changes over time.	Regression pass rate after model or prompt changes.
Usage	Whether users are adopting the capability.	Active users, sessions and repeat usage.

5.10.2. Feedback capture design

Feedback type	Capture method	Use
Rating	Thumbs up / down or simple score.	Identify weak answers and trends.

Reason category	Wrong number, unclear answer, missing caveat, too slow, no answer.	Route issues to the right owner.
Free-text comment	Optional user explanation.	Understand edge cases and user frustration.
Correction	User or SME provides corrected answer.	Add to evaluation set or backlog.
Escalation request	User asks for analyst or owner review.	Trigger support or governance route.
Follow-up pattern	Repeated rephrasing or clarification.	Identify unclear answers or poor grounding.

5.10.3. Monitoring dashboard outline

Area	Example measures	Owner
Adoption	Active users, sessions, interactions per user, repeat usage.	Product owner
Answer quality	Feedback score, correction rate, failed evaluation cases.	Evaluation owner
Retrieval quality	Retrieval failures, conflicting metadata, missing artefacts.	Data / semantic owner
Query safety	Validation failures, blocked queries, unsafe query attempts.	Evaluation / governance owner
Security and exposure	Permission failures, blocked sensitive fields, audit exceptions.	Security / governance lead
Performance	Latency, timeout rate, query duration, model-call duration.	AI / platform owner
Cost	Cost per interaction, model spend, query execution cost, logging cost.	Product / operating owner
Operations	Incidents, escalations, unresolved issues, SLA breaches.	Operating owner

5.10.4. Issue triage and routing

Issue type	Likely owner	Typical action
Wrong metric selected	Data / semantic owner	Fix metadata, synonyms or retrieval rules.
Wrong SQL / query	AI / solution architect	Improve query generation or validation.
Unsafe query blocked	Evaluation / governance owner	Review rule and confirm expected behaviour.
Missing caveat	Data / semantic owner	Update metadata and answer rules.
Permission issue	Security / governance lead	Fix access rule, identity mapping or masking.
Slow answer	AI / platform owner	Optimise model, retrieval, query or caching.
High cost	Product / operating owner	Adjust routing, limits, caching or usage controls.
Poor user understanding	Product owner / business SME	Improve answer format or guidance.
Repeated refusal	Product owner / data owner	Decide whether scope should expand or stay restricted.

5.10.5. Improvement triggers

Trigger	Example threshold	Response
Low feedback score	Feedback below agreed threshold for a question type.	Review answers and update backlog.
High correction rate	Repeated SME corrections for same metric or domain.	Add regression tests and fix root cause.
Retrieval failure spike	Missing metadata increases after source change.	Review metadata contract and indexes.
Validation failure spike	Many queries blocked for same rule.	Check prompt, metadata or rule design.
Refusal spike	Sudden increase in unsupported questions.	Review scope, user guidance or backlog.
Latency breach	95th percentile above target.	Optimise routing, query design or caching.
Cost breach	Cost per interaction above threshold.	Adjust model routing or query limits.
Security event	Restricted data appears in prompt, log or answer.	Stop affected route and escalate.
Regression failure	Golden test set falls below threshold.	Block release or roll back change.

5.10.6. Improvement backlog

Backlog item	Source signal	Root cause	Owner	Priority	Target action
	User feedback / validation / regression / incident				
	User feedback / validation / regression / incident				
	User feedback / validation / regression / incident				

5.10.7. Minimum evidence before closing Activity 5.10

- Before closing this activity, the team should be able to show:
- agreed quality signals;
- feedback capture design;
- monitoring dashboard outline;
- issue triage and ownership model;
- improvement triggers and thresholds;
- backlog and remediation process;
- governance link for model, prompt, metadata and validation changes.

5.11. Orchestration governance and change-control worksheet

5.11.1. 5.11.1. Change-control scope

Change area	Example change	Approval owner	Evidence required
Model	Change SQL-generation model.	AI / solution architect	Evaluation results, latency/cost impact, risk review.
Prompt	Update answer-generation prompt.	AI / solution architect	Regression results, prompt version, approval note.
Tool	Add a new execution tool.	Security / governance lead	Permission review, logging design, failure behaviour.
Metadata contract	Add new fields to metric cards.	Data / semantic owner	Updated contract, version history, downstream impact.
Validation rule	Relax row-limit or join rule.	Evaluation / governance owner	Failure analysis, risk review, test evidence.
Threshold	Change cost, latency or quality threshold.	Product / operating owner	Impact assessment and monitoring plan.
Safe-failure rule	Change refusal or escalation rule.	Product / governance owner	User impact review and risk assessment.
Logging / retention	Change trace or prompt retention.	Security / governance lead	Privacy, audit and retention review.

5.11.2. 5.11.2. Change classification

Change type	Risk level	Example	Required route
Minor configuration	Low	Update non-material wording in answer template.	Record change and test affected examples.
Prompt or routing change	Medium	Change model used for metadata ranking.	Regression test and owner approval.
Validation change	Medium / high	Change join, row-limit or cost rule.	Risk review, test evidence and approval.
Tool permission change	High	Allow a tool to access new data source.	Security review and formal approval.
Security / retention change	High	Store prompts for longer period.	Privacy, security and governance review.
Scope expansion	High	Add new domain, metric group or user group.	Product, data, security and evaluation review.

5.11.3. 5.11.3. Release evidence checklist

Evidence item	Required for	Owner
Change description	All changes	Change owner
Reason for change	All changes	Change owner
Impacted flow step	All changes	AI / solution architect
Prompt / model / tool version	Relevant technical changes	AI / solution architect
Regression test result	Prompt, model, tool, validation and metadata changes	Evaluation owner
Security review	Tool, access, exposure, retention and logging changes	Security / governance lead

Cost and latency impact	Model, routing, query and tool changes	Product / operating owner
Rollback plan	Medium and high-risk changes	AI / solution architect
Approval record	Medium and high-risk changes	Governance owner

5.11.4. 5.11.4. Regression and rollback triggers

Trigger	Example	Required response
Answer quality drop	Golden test set score falls below threshold.	Block release or roll back.
Safe-failure failure	Unsupported question is answered.	Stop affected route and fix rule/prompt.
Security issue	Restricted data appears in prompt, log or answer.	Disable route and escalate.
Cost spike	Cost per interaction exceeds threshold.	Review routing, model choice and query limits.
Latency breach	Response time exceeds user-journey target.	Optimise, restrict or roll back.
Validation failure spike	More queries blocked or failing after change.	Review prompt, metadata or validation rule.
User feedback issue	Increase in wrong or unclear answer reports.	Triage and add regression cases.

5.11.5. 5.11.5. Governance cadence

Cadence	Purpose	Participants
Weekly during prototype	Review issues, failed tests, design changes and risks.	Product, AI architect, data owner, evaluation owner.
Fortnightly during pilot	Review quality, usage, cost, feedback and change backlog.	Product, operating owner, governance, business SME.
Monthly after launch	Review performance, incidents, changes and improvement priorities.	Product, data, AI, security and operating owners.
Ad hoc	Review high-risk changes, incidents or security events.	Relevant decision owners and approvers.

5.11.6. 5.11.6. Minimum evidence before closing Activity 5.11

Before closing this activity, the team should be able to show:

- defined change-control scope;
- approval owners by change type;
- release evidence requirements;
- regression and rollback triggers;
- governance cadence;
- decision and approval logging route;
- unresolved governance risks and owners.